Work Explanation & Workflow Logic

n8n Automation Assessment

Candidate	**Sukhshum Vaishnavi**
Platform	n8n

1. Overview

This submission contains an automated GitHub Trending Digest workflow, a separate error-handling workflow, and an additional uptime-monitoring workflow. The workflows were designed in n8n and use Discord webhooks to deliver notifications.

2. Main Workflow — GitHub Trending Digest

The main workflow finds recently created machine-learning repositories on GitHub, identifies the five most-starred results, enriches the leading repository with README content, and sends a digest to a Discord channel.

1. **Trigger:** The workflow can start from an hourly Schedule Trigger or a Webhook Trigger used for manual testing.
2. **Fetch repositories:** An HTTP Request node calls the GitHub repository search API. The query searches for repositories with the machine-learning topic created within the previous seven days.
3. **Transform and rank:** A Code node extracts repository name, owner, URL, star count and description, sorts results by stars, and keeps the top five.
4. **Select and validate:** The next Code and IF nodes select the highest-ranked repository and check that valid repository data exists before README enrichment.
5. **README enrichment:** A second HTTP Request node retrieves the selected repository's README. A fallback path is provided if that request fails.
6. **Conditional formatting:** An IF node checks whether the selected repository has more than 1,000 stars and routes it to either hot-repository or normal-digest formatting.
7. **Discord notification:** The formatted digest is sent to Discord using a configured Discord webhook credential.

The main workflow was manually tested using its webhook test URL. The observed execution successfully fetched GitHub data, selected five repositories, retrieved the leading repository's README, followed the normal-digest branch for the tested repository, and delivered the notification to Discord.

3. Error-Handling Workflow

A separate workflow named “Morning Brief - Error Handler” contains an Error Trigger connected to a Discord notification node. It is intended to receive unhandled failures from the main workflow and send an alert containing workflow information, the last executed node, error details and a timestamp. It is linked through the main workflow's Settings → Error Workflow option.

The main workflow also has an inline handled-error path for cases such as a failed GitHub search request or no matching repositories. That path sends a no-data alert. It is separate from the Error Trigger workflow, which handles unhandled execution failures.

4. Bonus Workflow — Uptime Monitor

The bonus workflow independently monitors the public RealWorld Conduit demo application used during Task 1. Its configured target is https://demo.realworld.show/. It is kept separate from the GitHub digest so the monitoring schedule and failures do not interfere with the digest workflow.

8. **Scheduled check:** A Schedule Trigger is configured to run the health check every five minutes.
9. **HTTP request:** An HTTP Request node sends a GET request to the configured application URL, with an 8-second timeout and a retry policy of up to three attempts with a two-second wait between attempts.
10. **Response analysis:** A Code node records the target URL, response time, status code and whether the response is considered healthy. HTTP 200 is treated as healthy; a missing response or another status is treated as unavailable.
11. **Conditional alert:** An IF node checks the Boolean up value. A healthy result follows the no-action branch, while an unhealthy result sends a downtime alert to Discord.
12. **Daily summary:** A separate daily Schedule Trigger is configured for 8:00 AM. A Code node builds a summary from checks retained in n8n workflow static data, and a Discord node sends it.

5. Testing Performed

Workflow / Test	Observed result	Status
Main GitHub digest	GitHub search, transformation, README enrichment and Discord delivery completed successfully in a manual webhook-triggered execution.	Passed
Uptime — unavailable target	An intentionally invalid URL produced no HTTP response; the workflow marked the application unavailable and sent a Discord downtime alert.	Passed
Uptime — healthy target	The Conduit demo URL was restored and the healthy branch was tested; the no-action branch was followed.	Passed
Error Handler	The separate Error Handler was configured and linked to the main workflow. A production-triggered failure test should be performed and recorded before claiming end-to-end validation.	Linked; runtime test to confirm
Daily uptime summary	A daily summary node and 8:00 AM schedule are present. Persistent history should be verified after activated production executions.	Production verification pending

6. Configuration and Operational Notes

- Discord notifications use a webhook credential configured in n8n. Webhook URLs and other secrets are not included in this document.
- The main workflow has both an hourly schedule trigger and a webhook trigger for testing.
- The uptime monitor uses the public Conduit demo as an external monitoring target; it is not an application owned or deployed by the candidate.
- The uptime monitor's daily summary uses n8n workflow static data. Manual editor executions should not be treated as proof that this history persists; verify the summary using successful production-triggered executions.
- The workflows must be saved and activated/published in n8n for production schedules to run. The local n8n process and host machine must remain available while monitoring is expected to run.

7. Deliverables

The submission package contains the main GitHub Trending Digest workflow JSON, the separate Error Handler workflow JSON, the bonus Uptime Monitor workflow JSON, this explanation document, and execution screenshots. A Loom walkthrough or equivalent screen recording can be shared alongside these files if requested.